

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_095035a0-e9f\agent-a54c2441e0a3440ee.jsonl`
- SHA-256: `e2fdb462677f7bfff6809c907189bb0d14da9c0fafe09c3f1b66dc6c982e4ead2`
- Source modified: `2026-06-13T19:14:42+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf\_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:13:53.196Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"Default fusion only matches wasb seeding when indexing.wasb.velocity_threshold is left at its default","severity":"low","file":"backend/pipeline/segmenters/trajectory_hybrid.py:149","detail":"The windowing threshold is read family-scoped: velocity_thresh = idx_cfg.get(self.family, {}).get('velocity_threshold', 0.02). For wasb_hybrid family='wasb' (reads indexing.wasb.velocity_threshold); for FusionSegmenter family='fusion' (reads indexing.fusion.velocity_threshold). Both default to 0.02 (FusionConfig at models.py:89 vs WasbIndexConfig at models.py:69), so the DEFAULT-config claim ('seeds identically to its wasb substrate') is literally true and the test exercises it.\n\nHowever, the runner config (weights/distro/etc.) is read from indexing.wasb regardless of family (fusion_hybrid.py:67 -> WasbConfig.from_indexing_cfg reads idx_cfg['wasb']), while the windowing threshold is read from indexing.fusion. So if a user has TUNED indexing.wasb.velocity_threshold away from 0.02 (a documented per-video calibration knob per wasb_hybrid.py:7-9), FusionSegmenter will still window at fusion's 0.02 and diverge from that tuned wasb run -- a config skew, not a code regression. The FusionConfig docstring (models.py:84-86) flags this ('set equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run'), so it is intentional, but it is a real footgun for anyone comparing default-fusion to an already-tuned wasb run. The literal NO-REGRESSION claim as scoped (default config) is unaffected.", "fix":"None required for the claim. Optionally, when documenting/promoting fusion, note that an A/B against a tuned wasb baseline must copy indexing.wasb.velocity_threshold into indexing.fusion.velocity_threshold (the docstring already says this; consider asserting/warning at config load if the two differ while substrate='wasb')."}]

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:55.894Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:56.856Z)

```
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,
160             "merge_gap": merge_gap,
161             "min_action_window_duration": min_window_duration,
162             **runner_params,
163         }
164
165         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166         ok, msg = runner.healthcheck()
167         if not ok:
168             logger.error("%s WSL environment not ready: %s", label, msg)
169             return []
170         logger.info("%s WSL env OK: %s", label, msg)
171
172         # --- Phase 2: trajectory -> candidate rally windows ---
173         # Trajectory detectors process the whole video in one pass (they carry
174         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175         t_start = time.time()
176         out_dir = os.path.join("output", self.family)
177         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178         if not csv_path:
179             logger.error("%s produced no trajectory; aborting.", label)
```

4. user (2026-06-13T19:13:58.276Z)

```
55     """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58     these knobs actually take effect ? previously this whole block was silently dropped
59     because nested config sections defaulted to extra='ignore'.
60     """
61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66     sport: str = "badminton"
67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73     keep_frames: bool = False
74     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75     min_free_gb: float = 0.0
76
77
78     class FusionConfig(BaseModel):
79         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81         Every default reproduces control behaviour: the fusion segmenter persists the
82         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83         true ? the person-cue features, but it does NOT gate the served handoff unless a
84         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86         supplied ? off-court persons (spectators / adjacent courts) are excluded.
87         """
88         # Which trajectory substrate seeds the windows (shuttle stays primary).
89         substrate: Literal["wasb", "tracknet"] = "wasb"
90         # Windowing velocity threshold for the fusion family (the engine reads
91         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93         velocity_threshold: float = 0.02
94         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95         # enabled ? so the default path never imports torch / touches the GPU.
96         cue_flags: dict[str, bool] = Field(default_factory=dict)
97         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99         min_crossings: int = Field(default=0, ge=0)
100         # Served Gate B: when the person cue is on, drop windows with median in-court
players
101         # < this. 0 = OFF.
102         min_players: int = Field(default=0, ge=0)
103         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104         court_polygon: Optional[list[list[float]]] = None
105         # Net line for the person two-sided-motion split (person features only ? the shuttle
106         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
```

```

107         net_axis: Literal["x", "y"] = "y"
108         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111     class IndexingConfig(BaseModel):
112         default_segmenter: str = "yolo_hybrid"
113         use_gpu: bool = True
114         motion_threshold: float = 0.08

```

5. user (2026-06-13T19:13:58.455Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13    never imports torch / touches the GPU.
14    - ``_select_for_handoff`` ? optional served Gate A/B: when
15    ``indexing.fusion.min_crossings``
16    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17    are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
20    experiment
21    harness can re-score any variant.
22
23    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24    but
25    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26    seeds
27    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28    default
29    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30    through the
31    experiment harness (EXPERIMENT_HARNESS.md), never silently.
32    """
33    from typing import Any, Dict, List, Optional, Tuple
34
35    from backend.pipeline.segmenters.base import segmenter_registry
36    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37    from backend.pipeline.detectors.base import DetectorRunner
38    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40    from backend.pipeline.detectors import rally_gate
41    from backend.pipeline.detectors import fusion_features as ff
42
43    class FusionSegmenter(TrajectoryHybridSegmenter):
44        """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45        B)."""
46
47        family = "fusion"
48        display_label = "Fusion"
49
50        def __init__(self, db: Any, config: Any):
51            super().__init__(db, config)
52            # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53            default path).

```

```

46         self._person: Optional[Any] = None
47
48     @property
49     def name(self) -> str:
50         return "fusion_hybrid"
51
52     @property
53     def description(self) -> str:
54         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                 "AI provider sets semantic boundaries.")
57
58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                frame_width: float, video_path: str,
72                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                          frame_width: float, video_path: str,
87                          fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)

```

```

98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109    def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                           window_stats: List[Dict[str, Any]],
111                           idx_cfg: Dict[str, Any]) -> List[int]:
112        """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113        (0) keep every window ? identical to the substrate. Persisted candidates are
114        untouched (full superset)."""
115        fcfg = idx_cfg.get("fusion", {})
116        min_crossings = int(fcfg.get("min_crossings", 0))
117        min_players = int(fcfg.get("min_players", 0))
118        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120        keep: List[int] = []
121        for i, st in enumerate(window_stats):
122            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                continue
124            if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                continue
126            keep.append(i)
127        return keep
128
129
130    segmenter_registry.register(FusionSegmenter)
131

```

6. assistant (2026-06-13T19:14:02.096Z)

Now let me verify the key claims. Let me check `WasbConfig.from\_indexing\_cfg` to confirm it reads `indexing.wasb` regardless of family, and check the `wasb\_hybrid.py` docstring about the velocity_threshold being a per-video calibration knob.

7. user (2026-06-13T19:14:03.632Z)

```

45:    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46:        w = (idx_cfg or {}).get("wasb", {}) or {}
47:        return cls(
48:            distro=w.get("wsl_distro", "Ubuntu"),
49:            repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50:            conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51:            conda_env=w.get("conda_env", "wasb"),
52:            weights_path=w.get("weights_path",
53:                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54:            sport=w.get("sport", "badminton"),
55:            wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56:            timeout_sec=int(w.get("timeout_sec", 1800)),
57:            keep_frames=bool(w.get("keep_frames", False)),
58:            min_free_gb=float(w.get("min_free_gb", 0.0)),
59:        )
60-
61-
62-class WasbRunner(WslCommandMixin, DetectorRunner):
63:    def __init__(self, cfg: WasbConfig):

```

```

64-         self.cfg = cfg
65-
66-     def healthcheck(self) -> Tuple[bool, str]:
67-         """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68-         cmd = (f"conda activate {self.cfg.conda_env} && "
69-               f"python -c \"import torch; print('torch', torch.__version__, "
70-               f"'cuda', torch.cuda.is_available())\"")

```

8. user (2026-06-13T19:14:04.420Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses WASB (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video calibration goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
34             badminton "
35             "weights) for precise candidate rallies + AI provider for semantic
36             boundaries.")
37
38     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
39     Any]]:
40         cfg = WasbConfig.from_indexing_cfg(idx_cfg)
41         return WasbRunner(cfg), {"weights_path": cfg.weights_path}

```

9. assistant (2026-06-13T19:14:21.662Z)

All claims verified against the code. Let me also confirm the `family` attribute resolution and that `velocity\_threshold` is NOT consumed by `WasbConfig.from\_indexing\_cfg` (so the runner config path indeed ignores it while the windowing path reads it family-scoped).

I've confirmed the key facts:

1. **Line 149** (`trajectory_hybrid.py`): `velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)` ? family-scoped read, default 0.02. For `fusion_hybrid`, `family = "fusion"` (`fusion_hybrid.py:40`), so it reads `indexing.fusion.velocity_threshold`. For

`wasb\_hybrid`, `family = "wasb"` (wasb_hybrid.py:24), reads `indexing.wasb.velocity\_threshold`. ?

2. **Both defaults are 0.02**: `WasbIndexConfig.velocity\_threshold = 0.02` (models.py:69) and `FusionConfig.velocity\_threshold = 0.02` (models.py:93). ?

3. **Runner config read from `indexing.wasb` regardless of family**:
`FusionSegmenter.\_build\_runner` (fusion_hybrid.py:67) calls
`WasbConfig.from\_indexing\_cfg(idx\_cfg)`, which reads `idx\_cfg.get("wasb", {})`
(wasb_runner.py:46), NOT `idx\_cfg.get("fusion")`. Critically, `from\_indexing\_cfg` does NOT
consume `velocity\_threshold` at all (lines 47-58) ? it's a runner/WSL config, while
velocity_threshold is purely a windowing knob read separately at line 149. ?

4. **`velocity\_threshold` is documented as a per-video calibration knob** (wasb_hybrid.py:7-9). ?

5. **The docstring at FusionConfig (models.py:91-92) explicitly flags this**: "set equal to
indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run." ?

Now assessing the finding's actual claim. The finding's title says default fusion only matches
wasb seeding when `indexing.wasb.velocity\_threshold` is left at default. This is literally true:
with default config, both are 0.02 and seeding is identical (so the no-regression claim under
review holds). The "footgun" is real but config-skew, not a code regression ? and it is
explicitly documented as intentional. The finding itself concludes "None required for the claim"
and rates it **low**. It is essentially a confirmation that the reviewed no-regression claim
holds, with an honest caveat about a documented, intentional A/B footgun.

This is accurate to the code but is a low-severity nitpick about a documented, intentional
design choice ? not an actual defect. The finding does not identify a bug; it confirms the claim
and notes a documented footgun.

10. user (2026-06-13T19:14:32.896Z)

Structured output provided successfully

11. assistant (2026-06-13T19:14:42.342Z)

I've completed the adversarial verification. The finding's factual claims all hold against the
code, but I'm returning **real=false** because it does not describe an actual defect ? it
confirms the reviewed no-regression claim and flags a documented, intentional design choice as a
footgun.

Key files verified (all absolute paths):

- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/trajectory\_hybrid.py:149` ? family-scoped windowing read
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/fusion\_hybrid.py:40,67` ? `family="fusion"`, runner built via `WasbConfig.from\_indexing\_cfg`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/wasb\_hybrid.py:7-9,24` ? per-video calibration knob, `family="wasb"``
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py:69,91-93` ? both defaults 0.02; docstring explicitly flags the skew as intentional
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb\_runner.py:45-59` ? `from\_indexing\_cfg` reads `indexing.wasb` and does NOT consume `velocity\_threshold`

The finding is self-admittedly "a config skew, not a code regression," recommends "None required for the claim," and the flagged behavior is explicitly documented as the intended A/B knob in the FusionConfig docstring. That makes it a low-severity nitpick that confirms rather than refutes the claim under review.